OCR H446 / PAPER 2 / UNIT 2.2.1

Programming techniques

CHAPTER CONTENTS

Select a topic to jump to its original worked pages.

01 Sequence and selection	8
02 Iteration and loop bounds	12
03 Recursion	16
04 Scope	20
05 Modularity	24
06 Parameter passing	28
07 IDE debugging	32
08 OOP	36
Parameter passing: byVal and byRef	41
2D arrays: indices and boundaries	45
File handling: read, write and close	49
OOP: constructors and inheritance	53

MASTER CONTENTS

Choose a learning cycle

Each topic has one explanation page, a worked trace, original exam-style practice and a separate self-marking page. Click any row to jump there.

2.2.1(a)	Sequence And Selection	pp. 8-11
2.2.1(a)	Iteration And Loop Bounds	pp. 12-15
2.2.1(b)	Recursion And The Call Stack	pp. 16-19
2.2.1(c)	Local And Global Scope	pp. 20-23
2.2.1(d)	Modularity And Subroutines	pp. 24-27
2.2.1(d)	Value Vs Reference	pp. 28-31
2.2.1(e)	Ide And Debugging	pp. 32-35
2.2.1(f)	Object-Oriented Design	pp. 36-39
2.2.2(a,b)	Recognise A Solvable Problem	pp. 40-43

NAVIGATION

Use the bookmark panel or linked rows. Separate questions and answers support independent study. Section 2.2.1 precedes 2.2.2.

MASTER CONTENTS

Choose a learning cycle

Each topic has one explanation page, a worked trace, original exam-style practice and a separate self-marking page. Click any row to jump there.

2.2.2(c)	**Problem Decomposition**	pp. 44-47
2.2.2(d)	**Divide And Conquer**	pp. 48-51
2.2.2(e)	**Abstraction In Design**	pp. 52-55
2.2.2(f)	**Backtracking**	pp. 56-59
2.2.2(f)	**Data Mining**	pp. 60-63
2.2.2(f)	**Heuristics**	pp. 64-67
2.2.2(f)	**Performance Modelling**	pp. 68-71
2.2.2(f)	**Pipelining**	pp. 72-75
2.2.2(f)	**Visualisation**	pp. 76-79

NAVIGATION

Use the bookmark panel or linked rows. Separate questions and answers support independent study. Section 2.2.1 precedes 2.2.2.

INDEPENDENT REVISION

How to use each four-page cycle

01

UNDERSTAND

Read the explanation, inspect the original diagram, and recall the highlighted key terms.

02

MODEL

Follow each change of value, branch or state; explain why the step is necessary.

03

TRY

Attempt all questions independently in black pen before consulting answers.

04

CHECK

Self-mark using indicative points; correct a weakness in green pen and explain the fix.

These are original RHSC practice questions, not OCR past-paper questions.

2.2 / PROBLEM SOLVING AND PROGRAMMING

Understand it. Model it. Try it. Check it.

Programming techniques and computational methods work together. Choose a clear model, trace its state, justify each algorithmic decision and evaluate constraints.

2.2.1 Programming techniques

Sequence, selection, iteration; recursion; scope; modularity, parameter passing; IDE; OOP

2.2.2 Computational methods

Computability, recognition, decomposition, divide-and-conquer, abstraction, six problem-solving methods

18 learning cycles: eight in 2.2.1 and ten in 2.2.2. Use the topic checklists to plan revision.

2.2 / SPECIFICATION CHECKLIST

Check every OCR requirement

Check each cycle only when you can apply it to an unfamiliar scenario. The OCR requirements are summarised, not reproduced verbatim.

SPEC		PAGES
2.2.1(a)	Sequence, iteration and branching; trace loops	8-11
2.2.1(a)	Guard and termination; FOR, WHILE, REPEAT	12-15
2.2.1(b)	Recursion, base cases and iterative comparisons	16-19
2.2.1(c)	Local and global variables; scope	20-23
2.2.1(d)	Modularity, functions and procedures	24-27
2.2.1(d)	Passing parameters by value and reference	28-31
2.2.1(e)	Use an IDE to develop and debug	32-35
2.2.1(f)	Object-oriented techniques	36-39
2.2.2(a,b)	Solvable features; problem recognition	40-43

OFFICIAL SOURCE

Cambridge OCR H446 A Level Computer Science, specification v3.0 (April 2026), section 2.2.1 and 2.2.2.
Use these checklists to choose a topic, then attempt its exam-style questions.

2.2 / SPECIFICATION CHECKLIST

Check every OCR requirement

Check each cycle only when you can apply it to an unfamiliar scenario. The OCR requirements are summarised, not reproduced verbatim.

SPEC		PAGES
2.2.2(c)	Problem decomposition	44-47
2.2.2(d)	Divide and conquer	48-51
2.2.2(e)	Abstraction applied to problems	52-55
2.2.2(f)	Backtracking	56-59
2.2.2(f)	Data mining	60-63
2.2.2(f)	Heuristics	64-67
2.2.2(f)	Performance modelling	68-71
2.2.2(f)	Pipelining	72-75
2.2.2(f)	Visualisation	76-79

OFFICIAL SOURCE

Cambridge OCR H446 A Level Computer Science, specification v3.0 (April 2026), section 2.2.1 and 2.2.2.
Use these checklists to choose a topic, then attempt its exam-style questions.

2.2.1(a) / SEE IT / CONCEPT AND VISUAL

Choose an action from a testable condition

Sequence executes instructions in order. Branching selects a path after a Boolean test; both are fundamental program constructs.

SEQUENCE

One statement follows another.

IF

Execute only when a condition is true.

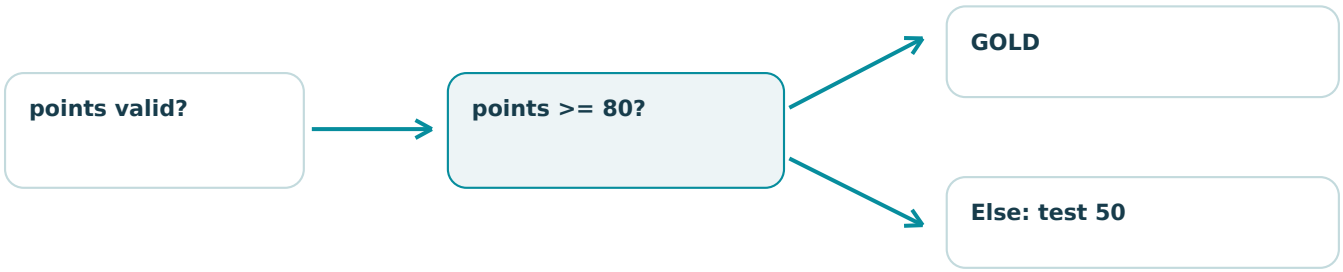
ELSE

Alternative path for a false condition.

TRACE

State exactly which statements execute.

HOW THE PARTS FIT TOGETHER



FROM THE DIAGRAM TO EXAM LANGUAGE

Guard validation before classification. ELSE is tied to the preceding IF, and comparison uses a Boolean result, not an assignment.

WORDS THAT EARN MARKS

Sequence	Ordered steps	**Selection**	Choose a branch
Boolean	True or false	**Condition**	Logical test

EXAM TRAP

ELSE IF conditions must be ordered carefully; changing the first threshold can change all later outcomes.

2.2.1(a) / WORKED EXAMPLE / STEP BY STEP

Trace a three-way classification with boundaries

A booking scheme awards GOLD for points >= 80, SILVER for points >= 50, otherwise BRONZE. Points must be between 0 and 100.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Read	**points = 80**	Start with the value to classify.
2 / Validate	**0 <= 80 <= 100: true**	Only classify an allowed value.
3 / First test	**80 >= 80: true**	Enter GOLD branch.
4 / Skip	**ELSE IF points >= 50 skipped**	Only one branch of this chain is taken.
5 / Assign	**award = GOLD**	Store selected result.
6 / Output	**Display GOLD**	The boundary 80 belongs to GOLD.

TRANSFER IT TO AN EXAM ANSWER

For points = 49 the outcome is BRONZE; for 50 it is SILVER. If points = 120, reject as invalid before allocating an award.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.2.1(a) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Define sequence, selection and Boolean condition.

Q2 / 5 MARKS

Trace the award for points 49, 50, 79 and 80. Explain why the tests must be ordered.

Q3 / 6 MARKS

Write pseudocode for this scheme, including rejection outside 0-100 and no award for invalid input.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.2.1(a) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Sequence means ordered execution [1]; selection chooses a path [1]; a condition evaluates TRUE/FALSE [1].

Q2 / 5

49 BRONZE [1], 50 SILVER [1], 79 SILVER [1], 80 GOLD [1]; test GOLD first so high values are not taken by the broad SILVER threshold [1].

Q3 / 6

Read points [1]; IF outside 0..100 reject [1]; ELSE IF >=80 choose GOLD [1]; ELSE IF >=50 choose SILVER [1]; ELSE choose BRONZE [1]; display outcome only for valid input [1].

COMMON MISCONCEPTION

ELSE IF conditions must be ordered carefully; changing the first threshold can change all later outcomes.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval: rhsccs.org/revision/h446/section-2-2-1/

2.2.1(a) / SEE IT / CONCEPT AND VISUAL

Use the right loop and protect termination

Iteration repeats a block. Count-controlled loops have known repeat counts; condition-controlled loops need a changing state and a termination condition.

FOR

Count-controlled repetition.

WHILE

Pre-condition: may run zero times.

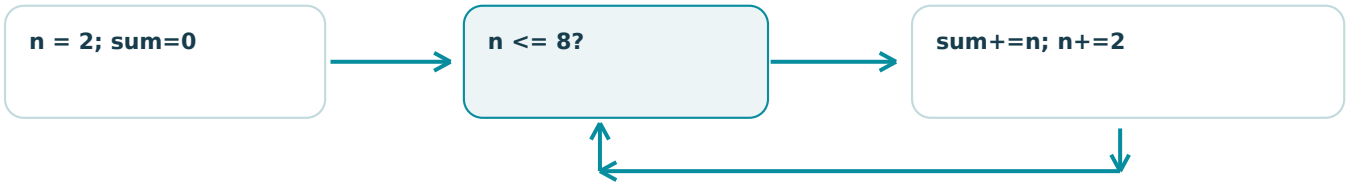
REPEAT

Post-condition: executes at least once.

INVARIANT

A fact that stays true each pass.

HOW THE PARTS FIT TOGETHER



FROM THE DIAGRAM TO EXAM LANGUAGE

In a WHILE loop, test before executing. A counter or other changing value must eventually make the loop condition false.

WORDS THAT EARN MARKS

- **Iteration** Repetition
- **Termination** Eventually stops
- **Loop guard** Continue condition
- **Accumulator** Running total

EXAM TRAP

A loop is not correct just because its body is correct: the guard, update and initial state must all be valid.

2.2.1(a) / WORKED EXAMPLE / STEP BY STEP

Trace an accumulator, including the final failed test

Sum 2 + 4 + 6 + 8 with n initially 2, total initially 0, and WHILE n <= 8. Update total then add 2 to n.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Setup	**n=2; total=0**	No values accumulated yet.
2 / Pass 1	**total=2; n=4**	Add the first even integer.
3 / Pass 2	**total=6; n=6**	Add 4 to the accumulator.
4 / Pass 3	**total=12; n=8**	Add 6 to the accumulator.
5 / Pass 4	**total=20; n=10**	Add 8 then advance beyond the bound.
6 / Exit	**10 <= 8 is false; output 20**	The next guard test stops iteration.

TRANSFER IT TO AN EXAM ANSWER

For upper bound 1, the WHILE body executes zero times and total remains 0. A REPEAT ... UNTIL loop would run once before its condition is checked.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.2.1(a) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Distinguish FOR, WHILE and REPEAT loops.

Q2 / 5 MARKS

Show n and total after each pass for n=2,4,6,8. Explain the exit condition.

Q3 / 6 MARKS

Design a condition-controlled algorithm that sums positive inputs until a sentinel -1; reject other negative numbers and do not include the sentinel.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.2.1(a) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

FOR has a counted range [1]; WHILE tests first [1]; REPEAT tests after and runs at least once [1].

Q2 / 5

After passes: (4,2) [1], (6,6) [1], (8,12) [1], (10,20) [1]; condition n <= 8 then becomes false [1].

Q3 / 6

Start total=0 [1]; read input [1]; loop while input != -1 [1]; add only when input >= 0 [1]; reject any negative other than -1 [1]; read again and output final total [1].

COMMON MISCONCEPTION

A loop is not correct just because its body is correct: the guard, update and initial state must all be valid.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval: rhsccs.org/revision/h446/section-2-2-1/

2.2.1(b) / SEE IT / CONCEPT AND VISUAL

Base case first; shrinking problem second

Recursion is a procedure or function calling itself on a smaller instance. A valid base case prevents indefinite self-calls.

BASE CASE

An answer without another call.

RECURSIVE CASE

Invoke the same function on a smaller problem.

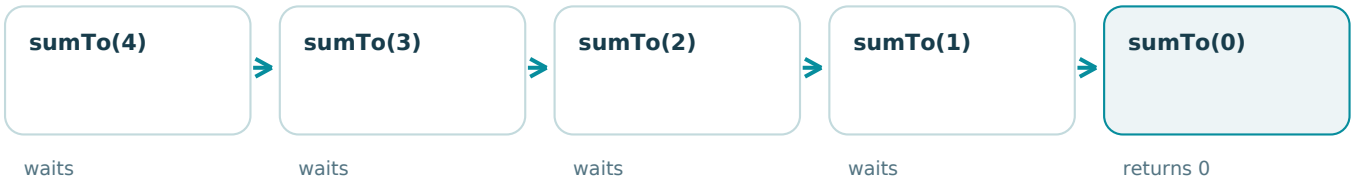
STACK FRAME

Keeps each call's local state.

RETURN

Results unwind to the caller.

HOW THE PARTS FIT TOGETHER



FROM THE DIAGRAM TO EXAM LANGUAGE

Each call waits for its inner call to return. Show the downward calls and then the upward returns; do not add numbers until the return stage.

WORDS THAT EARN MARKS

Recursion	Self-call	**Base case**	Stops calls
Call stack	Pending returns	**Stack overflow**	Too much depth

EXAM TRAP

Recursive calls do not automatically return in the same order they are made. Show both the call and return sequence.

2.2.1(b) / WORKED EXAMPLE / STEP BY STEP

Trace sumTo(4) with explicit return values

Function sumTo(n) returns 0 if n=0; otherwise it returns n + sumTo(n-1). All inputs in this model are non-negative integers.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Call	**sumTo(4) = 4 + sumTo(3)**	First return waits for next call.
2 / Call	**sumTo(3) = 3 + sumTo(2)**	Depth increases.
3 / Call	**sumTo(2) = 2 + sumTo(1)**	Another pending addition.
4 / Call	**sumTo(1) = 1 + sumTo(0)**	Approach the base case.
5 / Base	**sumTo(0) returns 0**	No further recursive call.
6 / Unwind	**1 -> 3 -> 6 -> 10**	Returns of sumTo(1..4) in order.

TRANSFER IT TO AN EXAM ANSWER

Iterative comparison: total=0; FOR k=1 TO 4, total=total+k. Both return 10; recursion keeps pending stack frames, while this loop uses constant extra space. Guard negative inputs.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.2.1(b) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

State the role of a base case and a stack frame. Give one recursion risk.

Q2 / 5 MARKS

Trace the call order and return values for sumTo(3), including sumTo(0).

Q3 / 6 MARKS

Write recursive pseudocode for factorial(n), n >= 0; explain its base case and termination, and compare its extra memory use with an iterative factorial.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.2.1(b) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Base case returns directly [1]; frame stores the pending caller state [1]; excessive recursion can overflow the stack [1].

Q2 / 5

Call sumTo(3), (2), (1), (0) [2]; returns 0, 1, 3, 6 [2]; every call decreases n [1].

Q3 / 6

Reject n<0 [1]; return 1 when n=0 [1]; otherwise return n * factorial(n-1) [1]; recursive input decreases [1] until the base case [1]; recursive stack depth grows with n, unlike an iterative accumulator with constant extra space [1].

COMMON MISCONCEPTION

Recursive calls do not automatically return in the same order they are made. Show both the call and return sequence.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval: rhsccs.org/revision/h446/section-2-2-1/

2.2.1(c) / SEE IT / CONCEPT AND VISUAL

Know which variable changes and which survives

Local variables belong to their procedure or function invocation. Global variables are available in a wider scope and can create unwanted coupling.

LOCAL

Declared inside a routine or block.

GLOBAL

Declared in wider program scope.

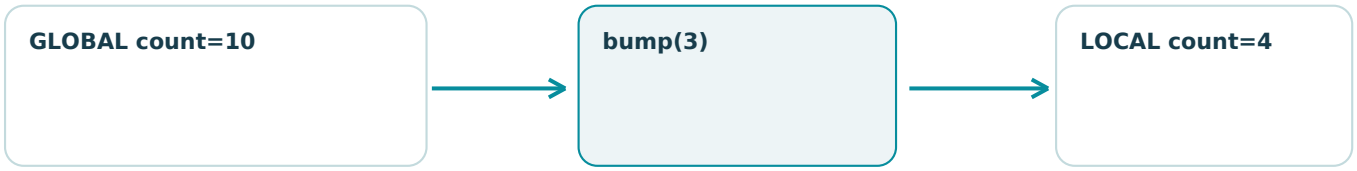
SHADOW

A local name can hide a global name.

SIDE EFFECT

A call may change shared state.

HOW THE PARTS FIT TOGETHER



Return 4 does not change the global count unless caller assigns it.

FROM THE DIAGRAM TO EXAM LANGUAGE

A local variable with the same name is not automatically the global. Separate each variable's lifetime and scope in a trace.

WORDS THAT EARN MARKS

- **Scope** Where name is valid
- **Local** Routine-owned value
- **Global** Shared program value
- **Shadowing** Name hides another

EXAM TRAP

Names that are spelled identically do not imply the variables share the same storage location.

2.2.1(c) / WORKED EXAMPLE / STEP BY STEP

Track two different variables called count

Global count starts at 10. Function bump(count) receives value 3 in its local parameter, increases it by 1, then returns it. The caller does not assign the result to global count.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Global	**count = 10**	Program-level value exists.
2 / Call	**bump(3)**	Create a new local parameter.
3 / Local	**count = 3**	The parameter hides the global name in function body.
4 / Update	**local count = 4**	Only the local parameter is incremented.
5 / Return	**function returns 4**	No global assignment occurs.
6 / End	**global count still 10**	Local parameter ceases to exist.

TRANSFER IT TO AN EXAM ANSWER

If the caller then executes count = bump(3), the global becomes 4 because the result is explicitly assigned.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.2.1(c) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Define local scope, global scope and shadowing.

Q2 / 5 MARKS

Trace the local return and global value after calling bump(3). Explain why they differ.

Q3 / 6 MARKS

Evaluate two possible designs for a score-tracking application: routines modify a global score, or functions return a new score. Justify which is easier to test.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.2.1(c) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Local: visible within a routine [1]; global: wider program visibility [1]; shadowing: local name masks wider one [1].

Q2 / 5

Parameter count starts 3 [1]; becomes 4 [1]; function returns 4 [1]; global remains 10 [1]; no global write takes place [1].

Q3 / 6

Global sharing can reduce parameter passing [1] but causes hidden dependencies [1] and unpredictable side effects [1]. Returning a new score makes inputs/outputs explicit [1], simplifies isolated tests [1], though values must be passed and assigned [1].

COMMON MISCONCEPTION

Names that are spelled identically do not imply the variables share the same storage location.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval: rhsccs.org/revision/h446/section-2-2-1/

2.2.1(d) / SEE IT / CONCEPT AND VISUAL

Break a program into reusable, testable routines

Modularity separates a solution into coherent components. Functions return values; procedures perform an action, possibly changing program state.

FUNCTION

Returns a computed value.

PROCEDURE

Performs an action without needing a return value.

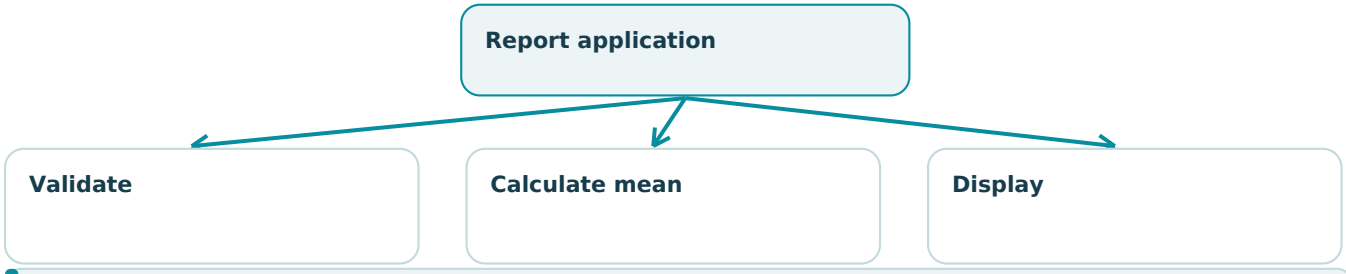
PARAMETER

Named input to a routine.

COHESION

Each routine has a clear job.

HOW THE PARTS FIT TOGETHER



FROM THE DIAGRAM TO EXAM LANGUAGE

Use distinct routine contracts to make testing precise. A validation function can be reused whenever a mark enters the system.

WORDS THAT EARN MARKS

Modularity	Split into routines	**Function**	Returns a value
Procedure	Performs an action	**Parameter**	Passed information

EXAM TRAP

A procedure is not defined by having no parameters; both procedures and functions may accept parameters.

2.2.1(d) / WORKED EXAMPLE / STEP BY STEP

Decompose a mark-reporting program into contracts

A mark sheet contains 60, 70, 80. A report must reject out-of-range marks, calculate the mean and display it.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Receive	**marks = [60,70,80]**	Main program obtains input.
2 / Validate	**validMarks(marks) => true**	Each mark is checked in 0..100.
3 / Sum	**sumMarks = 210**	Calculation is in one routine.
4 / Average	**average = 210 / 3 = 70**	Return a numeric result.
5 / Present	**showAverage(70)**	Procedure displays output.
6 / Test	**[60,120,80] => invalid**	Do not calculate from invalid data.

TRANSFER IT TO AN EXAM ANSWER

Be explicit about empty-list handling before division. A function returning a status or error can make failure visible to its caller.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.2.1(d) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Distinguish a function, a procedure and a parameter.

Q2 / 5 MARKS

Identify three routines for the report and give an input and output/result for each.

Q3 / 6 MARKS

Design modular pseudocode for validating, calculating and presenting a mean. Explain two testing advantages of this design.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.2.1(d) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Function returns a value [1]; procedure performs an action [1]; parameter supplies routine data [1].

Q2 / 5

Validation receives marks, returns Boolean [2]; average receives valid marks, returns mean [2]; display receives mean and outputs report [1].

Q3 / 6

Use separate validation routine [1], mean calculation [1], display routine [1]; refuse invalid/empty inputs [1]; test subroutines independently [1]; reuse validation or isolate a bug more quickly [1].

COMMON MISCONCEPTION

A procedure is not defined by having no parameters; both procedures and functions may accept parameters.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval: rhsccs.org/revision/h446/section-2-2-1/

2.2.1(d) / SEE IT / CONCEPT AND VISUAL

Trace copies and shared changes accurately

Pass-by-value gives a routine a separate value; pass-by-reference lets a routine operate on the caller's variable or storage.

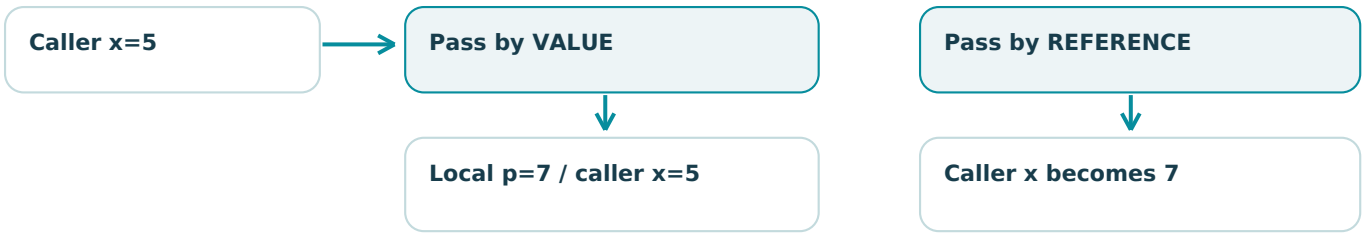
BY VALUE
Local changes do not alter caller scalar.

BY REFERENCE
Changes affect caller's original value.

CALLER
Owns the original variable.

SIDE EFFECT
A routine changes something outside itself.

HOW THE PARTS FIT TOGETHER



FROM THE DIAGRAM TO EXAM LANGUAGE

Different languages and data types have different semantics: the example uses explicit conceptual value/reference parameter modes, not Python list assignment rules.

WORDS THAT EARN MARKS

Pass by value	Use a copy	**Pass by reference**	Use original
Parameter	Routine input	**Side effect**	External change

EXAM TRAP

Passing a mutable object in one language does not automatically mean its parameter mechanism is pass-by-reference in the OCR sense.

2.2.1(d) / WORKED EXAMPLE / STEP BY STEP

Run the same routine with two passing modes

Procedure addTwo(p) sets p = p + 2. Caller variable x starts at 5. Compare calling it with parameter p BY VALUE versus p BY REFERENCE.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Caller	**x = 5**	Original variable.
2 / Value call	**p starts at 5**	Local copy is made.
3 / Value update	**p = 7**	Caller x remains 5.
4 / Reference call	**p aliases x**	Routine refers to original location.
5 / Reference update	**x becomes 7**	Change is visible to caller.
6 / Observe	**value mode x=5; reference x=7**	Same body, different parameter semantics.

TRANSFER IT TO AN EXAM ANSWER

A pure function returning x+2 could make the update explicit: x = addTwo(x). That differs from mutating the caller by reference.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.2.1(d) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Define pass-by-value, pass-by-reference and a side effect.

Q2 / 5 MARKS

Complete a trace of x=5 after addTwo(x) under each passing mode and explain the difference.

Q3 / 6 MARKS

Discuss when to use a by-reference parameter for a routine that updates both a balance and a transaction count. State testing risks.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.2.1(d) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Value passes a separate copy [1]; reference lets routine update original variable [1]; side effect changes state outside routine [1].

Q2 / 5

By-value caller x remains 5 [1]; local p becomes 7 [1]; by-reference caller x becomes 7 [1]; p refers to caller storage [1]; a later read sees changed state [1].

Q3 / 6

Reference can update balance [1] and transaction count [1] directly, but should validate transaction first [1]; unexpected side effects complicate tests [1]; two outputs can alternatively be returned explicitly [1]; document parameters and test no-change on failed operations [1].

COMMON MISCONCEPTION

Passing a mutable object in one language does not automatically mean its parameter mechanism is pass-by-reference in the OCR sense.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval: rhsccs.org/revision/h446/section-2-2-1/

2.2.1(e) / SEE IT / CONCEPT AND VISUAL

Use editor, breakpoints and watch variables deliberately

An integrated development environment combines authoring, executing, inspecting and debugging tools; an error can be located by observing state at a breakpoint.

EDITOR

Write and navigate source.

BREAKPOINT

Pause at a chosen location.

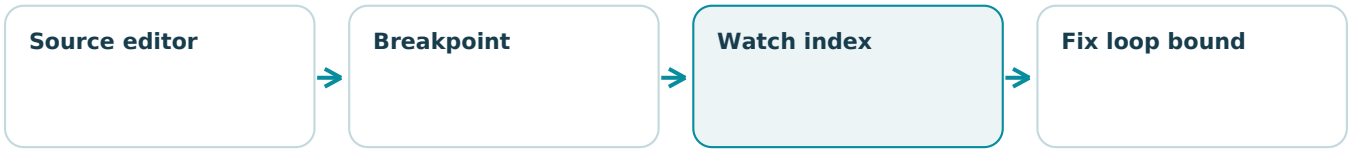
WATCH

Inspect values as execution proceeds.

STEPPING

Execute one statement at a time.

HOW THE PARTS FIT TOGETHER



FROM THE DIAGRAM TO EXAM LANGUAGE

A successful run with one input does not establish correctness. Retest boundary and erroneous inputs after changing the code.

WORDS THAT EARN MARKS

IDE	Development tools	**Breakpoint**	Pause execution
Watch window	Inspect state	**Step over**	Next statement

EXAM TRAP

A syntax-error-free program may still have logical and runtime errors.

2.2.1(e) / WORKED EXAMPLE / STEP BY STEP

Locate an off-by-one error with a watch window

An average-mark routine loops through indices 0 to LENGTH(marks) inclusive. For marks=[8,10,12], the array has three elements.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Expected	**Valid indices are 0,1,2**	Length 3 does not include index 3.
2 / Pause	**Breakpoint inside loop**	Stop just before array access.
3 / Watch	**index=0,1,2: valid**	Inspect counter and current element.
4 / Failure	**index=3: out of range**	Inclusive upper bound is wrong.
5 / Correct	**Use index < LENGTH(marks)**	Last visited index is 2.
6 / Retest	**Try empty, one-item, three-item arrays**	Check other boundaries and empty handling.

TRANSFER IT TO AN EXAM ANSWER

Use diagnostic messages or a call stack when the fault occurs inside nested routines. A debugger helps find faults; it does not replace deliberate tests.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.2.1(e) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Name three useful IDE debugging facilities.

Q2 / 5 MARKS

Explain how a breakpoint, step-through and watch window identify the indexing error.

Q3 / 6 MARKS

Propose a debugging and test plan for a function failing only on empty or one-item arrays.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.2.1(e) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Breakpoint [1]; watch variable window [1]; stepping, call stack or run tools [1].

Q2 / 5

Breakpoint just before array access [1]; step through [1]; inspect index [1]; find index=3 [1]; replace inclusive upper bound with < length [1].

Q3 / 6

Reproduce with failing input [1]; set breakpoints at loop/mean calculation [1]; inspect length/index/total [1]; guard against empty division [1]; test one-item, boundary and ordinary cases [1]; rerun whole relevant test suite after fix [1].

COMMON MISCONCEPTION

A syntax-error-free program may still have logical and runtime errors.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval: rhsccs.org/revision/h446/section-2-2-1/

2.2.1(f) / SEE IT / CONCEPT AND VISUAL

Connect a class, an object and its behaviour

Object-oriented programming encapsulates data and methods together. Classes define objects; inheritance and polymorphism support reusable behaviour.

CLASS

Blueprint for objects.

OBJECT

Instance with its own state.

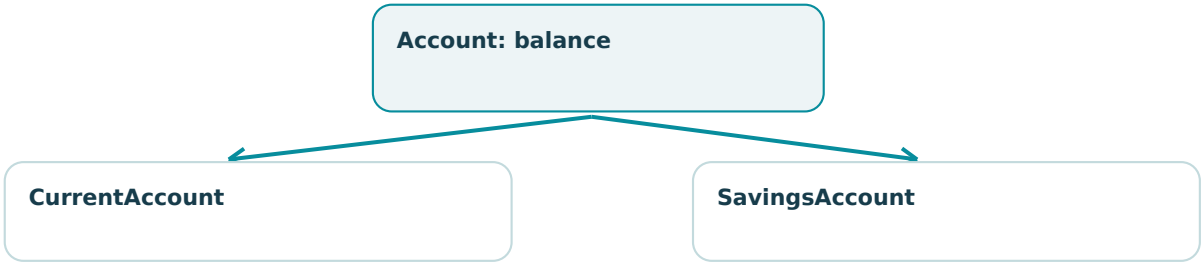
ENCAPSULATION

Protect state through controlled access.

INHERITANCE

Specialise shared features.

HOW THE PARTS FIT TOGETHER



FROM THE DIAGRAM TO EXAM LANGUAGE

Both current and savings accounts can share a valid deposit method. A savings account may override calculateInterest while preserving the shared interface.

WORDS THAT EARN MARKS

Class	Blueprint	**Object**	Instance
Method	Behaviour	**Polymorphism**	One interface; forms

EXAM TRAP

Inheritance represents an is-a relationship, not every has-a relationship. Do not confuse overloading and overriding.

2.2.1(f) / WORKED EXAMPLE / STEP BY STEP

Trace one account object with a protected balance

An Account object starts with balance 100. deposit(30) validates a positive amount; withdraw(50) requires enough funds. A later withdraw(90) is attempted.

STEP	VALUE / ACTION	WHY THIS STEP MATTERS
1 / Instantiate	a = Account(100)	Object owns its balance.
2 / Deposit	a.deposit(30)	Public method checks amount > 0.
3 / State	balance = 130	Method updates encapsulated state.
4 / Withdraw	a.withdraw(50)	Balance sufficient.
5 / State	balance = 80	Valid withdrawal changes state.
6 / Reject	withdraw(90) => rejected	Balance stays 80: invariant retained.

TRANSFER IT TO AN EXAM ANSWER

A child SavingsAccount class could reuse deposit and withdraw, and override interest calculation. Calling calculateInterest on different account types demonstrates polymorphism.

EXAM HABIT / Show the decision, the data or state change, and why it is correct.

2.2.1(f) / INDEPENDENT EXAM PRACTICE

Trace, explain and apply

Attempt each original exam-style question before viewing its separate answer page.

BLACK PEN / ANSWER FIRST. CHECK YOUR MARK POINTS ON THE NEXT PAGE.

Q1 / 3 MARKS

Distinguish a class, an object and encapsulation.

Q2 / 5 MARKS

Trace the balance and explain why the last withdrawal is rejected.

Q3 / 6 MARKS

Design a class hierarchy for Account, CurrentAccount and SavingsAccount, explaining inheritance, polymorphism and one limitation.

Answer with precise vocabulary, working and a conclusion where a question asks you to evaluate.

2.2.1(f) / SELF-MARKING / INDICATIVE ANSWERS

Mark, correct and explain

Use the indicative points to identify what is correct, then correct one weakness in green pen.

Q1 / 3

Class is blueprint [1]; object is instantiated state [1]; encapsulation controls access to state via methods [1].

Q2 / 5

After deposit 130 [1], after withdrawal 80 [1], insufficient funds for 90 [1], reject operation [1], balance remains 80 [1].

Q3 / 6

Account base class holds shared data/behaviour [1]; subclasses inherit common methods [1]; override appropriate method e.g. interest [1]; same call works with account subtypes (polymorphism) [1]; protect balance with validation [1]; deeper hierarchy may be harder to maintain [1].

COMMON MISCONCEPTION

Inheritance represents an is-a relationship, not every has-a relationship. Do not confuse overloading and overriding.

GREEN PEN / Correct an error, then explain why the corrected answer is right.

Further retrieval: rhsccs.org/revision/h446/section-2-2-1/

OCR APPENDIX 5d / APPLIED PROGRAMMING

Read it. Trace it. Write it.

Four additional four-page learning cycles complement the 18 existing Section 2.2 cycles. The examples use OCR H446 Appendix 5d conventions but are original practice, not OCR examination questions or prescribed code to memorise.

01 Passing parameters by value and reference
2.2.1(c,d) / pp. 2-5

02 Zero-based 2D arrays and loop bounds
2.2.1(a) / 1.4.2 / pp. 6-9

03 File read/write and end-of-file loops
Appendix 5d / synoptic / pp. 10-13

04 Constructors, inheritance and methods
2.2.1(f) / pp. 14-17

What OCR expects

Recognise the OCR reference notation, explain each state change, write a logically correct solution and justify boundary decisions. Alternative readable pseudocode or suitable programming-language solutions may also be acceptable.

What this adds

The chapter already covers concepts. This bridge adds complete code models, intermediate traces, original exam-style writing tasks, and separate Check pages.

Source

Cambridge OCR H446 specification v3.0 (2026), section 2.2 and Appendix 5d, including file and object-oriented conventions.

UNDERSTAND / 1 OF 4

Parameter passing: byVal and byRef

OCR assumes parameters are passed by value unless otherwise stated. When relevant it labels each parameter :byVal or :byRef. Trace the caller as well as the local subroutine state.

KEY IDEAS AND APPENDIX CONVENTIONS

byVal / copy

An update inside the procedure changes the local parameter, not the caller variable.

byRef / shared original

An update inside the procedure changes the caller variable immediately.

Scope

Local variables belong to the subroutine. A global value can persist and be changed elsewhere; explicit parameters reduce hidden dependencies.

EXAM-STYLE PSEUDOCODE

```
procedure adjust(a:byVal, b:byRef)
    a = a + 4
    b = b + a
endprocedure

x = 6
y = 6
adjust(x, y)
print(x)  // 6
print(y)  // 16
```

MODEL / 2 OF 4

Trace the caller and the parameter separately

A complete worked model: follow the code, then track each intermediate state before the final output.

```
procedure adjust(a:byVal, b:byRef)
    a = a + 4
    b = b + a
endprocedure

x = 6; y = 6
adjust(x,y)
print(x)
print(y)
```

WORKED TRACE / THINK ALOUD

1. Caller starts x=6 and y=6.

2. Inside adjust, a starts as a copy: 6; b refers to caller y.

3. a becomes 10. Updating b to 6+10 changes y to 16.

4. After return, the caller prints 6, 16. x did not change.

TRY / 3 OF 4

Trace, write, explain

Attempt the three original exam-style questions independently. The next page contains indicative marking points, so do not turn over until you finish.

BLACK PEN / ANSWER BEFORE TURNING THE PAGE

QUESTION 1 / 3 MARKS

State the difference between :byVal and :byRef and identify the default when it is not specified.

QUESTION 2 / 5 MARKS

Trace adjust(x,y) when x=3 and y=5. Show local a, final x and final y using the model code.

QUESTION 3 / 6 MARKS

Write procedure addCharge(bill:byRef, extra:byVal) that rejects a negative extra and otherwise adds extra to bill. Show a call and explain which variable changes.

CHECK / 4 OF 4

Self-mark and correct

Award only marks supported by your solution. Correct one error using green pen and explain what changed. The points below are not official OCR mark schemes.

GREEN PEN / SELF-MARK / INDICATIVE POINTS

Q1 / 3 MARKS

Value passes a copy [1]; reference can modify the caller variable [1]; the default is by value unless otherwise stated [1].

Q2 / 5 MARKS

Start a=3 and b refers to y=5 [1]; a becomes 7 [1]; b/y becomes 12 [1]; x stays 3 [1]; print 3,12 [1].

Q3 / 6 MARKS

Correct procedure name/parameters [1]; bill marked byRef [1]; extra byVal [1]; reject negative extra [1]; add extra to bill only otherwise [1]; correct call and explanation that the caller bill changes [1].

EXAMPLE ALGORITHM / AN ACCEPTABLE SOLUTION

```
procedure addCharge(bill:byRef, extra:byVal)
    if extra >= 0 then
        bill = bill + extra
    endif
endprocedure
amount = 20; addCharge(amount,5)  // 25
```

CORRECT ONE ERROR IN GREEN AND EXPLAIN THE CHANGE.

UNDERSTAND / 1 OF 4

2D arrays: indices and boundaries

Appendix 5d uses zero-based arrays, including [row,column] 2D notation. The loop limits must match the declared dimensions. Trace a concrete grid rather than assuming index 1 is the first cell.

KEY IDEAS AND APPENDIX CONVENTIONS

Dimensions

array grid[3,4] has three rows 0..2 and four columns 0..3.

2D indexing

grid[r,c] selects one value. In OCR-style examples, do not silently change to the Python form grid[r][c].

Nested loops

Outer row 0..2; inner column 0..3. This visits 12 elements. Check the final legal index.

EXAM-STYLE PSEUDOCODE

```
array grid[3,4]  // 3 rows, 4 columns
grid[0,0] = 7
grid[2,3] = 9
for r = 0 to 2
    for col = 0 to 3
        print(grid[r,col])
    next col
next r
```

MODEL / 2 OF 4

Count values in an entire grid

A complete worked model: follow the code, then track each intermediate state before the final output.

```
function countMatches(grid, target)
    count = 0
    for r = 0 to 2
        for col = 0 to 3
            if grid[r,col] == target then
                count = count + 1
            endif
        next col
    next r
    return count
endfunction
```

WORKED TRACE / THINK ALOUD

1. A 3x4 grid contains 12 indexed positions.

2. The first cell is [0,0], not [1,1].

3. After [0,3] the next cell visited is [1,0].

4. The last legal cell is [2,3]; [3,0] is out of bounds.

TRY / 3 OF 4

Trace, write, explain

Attempt the three original exam-style questions independently. The next page contains indicative marking points, so do not turn over until you finish.

BLACK PEN / ANSWER BEFORE TURNING THE PAGE

QUESTION 1 / 3 MARKS

Give the first and last valid row and column indexes for array grid[3,4].

QUESTION 2 / 5 MARKS

A 3 by 4 array has target values in positions [0,1], [1,3] and [2,0]. Trace countMatches and give its return value.

QUESTION 3 / 6 MARKS

Write OCR-style nested loops to set every cell of array board[2,5] to 0. Include both correct bounds and a brief explanation of why every cell is reached.

CHECK / 4 OF 4

Self-mark and correct

Award only marks supported by your solution. Correct one error using green pen and explain what changed. The points below are not official OCR mark schemes.

GREEN PEN / SELF-MARK / INDICATIVE POINTS

Q1 / 3 MARKS

Rows 0..2 [1]; columns 0..3 [1]; last valid position is [2,3] and first [0,0] [1].

Q2 / 5 MARKS

Initial count 0 [1]; visit rows 0,1,2 [1]; count increases at [0,1] [1], [1,3] [1], [2,0] and final return 3 [1].

Q3 / 6 MARKS

Outer row loop 0..1 [1]; inner column loop 0..4 [1]; write board[r,col]=0 [1]; correct nesting [1]; 2x5=10 cells [1]; no negative/out-of-range indexes [1].

EXAMPLE ALGORITHM / AN ACCEPTABLE SOLUTION

```
array board[2,5]
for r = 0 to 1
    for col = 0 to 4
        board[r,col] = 0
    next col
next r
```

CORRECT ONE ERROR IN GREEN AND EXPLAIN THE CHANGE.

UNDERSTAND / 1 OF 4

File handling: read, write and close

Appendix 5d shows file operations as methods on a file object: openRead, readLine, endOfFile, openWrite, writeLine and close. These conventions are useful in applied Paper 2 scenarios.

KEY IDEAS AND APPENDIX CONVENTIONS

Read a file

myFile=openRead("data.txt") then myFile.readLine() to retrieve text. Use an end-of-file test for a loop.

Write a file

myFile=openWrite("result.txt") then myFile.writeLine(text). Opening for writing replaces existing contents in the OCR model.

Finish safely

Close every opened file. Do not read after endOfFile() is reached.

EXAM-STYLE PSEUDOCODE

```
source = openRead("scores.txt")
while NOT source.endOfFile()
    line = source.readLine()
    print(line)
endwhile
source.close()

out = openWrite("status.txt")
out.writeLine("complete")
out.close()
```

MODEL / 2 OF 4

Copy each line without losing the final line

A complete worked model: follow the code, then track each intermediate state before the final output.

```
source = openRead("input.txt")
dest = openWrite("output.txt")
while NOT source.endOfFile()
    line = source.readLine()
    dest.writeLine(line)
endwhile
source.close()
dest.close()
```

WORKED TRACE / THINK ALOUD

1. For source lines "A" then "B", the first loop writes "A".
2. The second loop writes "B".
3. The following test finds endOfFile() true: loop ends.
4. Both files are closed; output has the two lines in order.

TRY / 3 OF 4

Trace, write, explain

Attempt the three original exam-style questions independently. The next page contains indicative marking points, so do not turn over until you finish.

BLACK PEN / ANSWER BEFORE TURNING THE PAGE

QUESTION 1 / 3 MARKS

Name the OCR-style operations used to open for reading, read one line and close a file.

QUESTION 2 / 5 MARKS

A file contains the lines Oak, Elm, Ash. Trace the model loop: how many times does it run and what appears in output.txt? Include the final end-of-file decision.

QUESTION 3 / 6 MARKS

Write OCR-style pseudocode that copies every line from notes.txt to backup.txt, converting none of the text, and closes both files. Explain one consequence of openWrite.

CHECK / 4 OF 4

Self-mark and correct

Award only marks supported by your solution. Correct one error using green pen and explain what changed. The points below are not official OCR mark schemes.

GREEN PEN / SELF-MARK / INDICATIVE POINTS

Q1 / 3 MARKS

openRead [1]; readLine [1]; close [1].

Q2 / 5 MARKS

Three loop iterations [1]; Oak copied [1]; Elm copied [1]; Ash copied [1]; endOfFile becomes true then loop stops, without a fourth read [1].

Q3 / 6 MARKS

Open notes with openRead [1]; backup with openWrite [1]; loop until endOfFile [1]; read and write each line [1]; close both files [1]; openWrite overwrites existing destination contents [1].

EXAMPLE ALGORITHM / AN ACCEPTABLE SOLUTION

```
f = openRead("notes.txt")
out = openWrite("backup.txt")
while NOT f.endOfFile()
    out.writeLine(f.readLine())
endwhile
f.close(); out.close()
```

CORRECT ONE ERROR IN GREEN AND EXPLAIN THE CHANGE.

UNDERSTAND / 1 OF 4

OOP: constructors and inheritance

OCR Appendix 5d models constructors as new, subclasses with inherits and calls to the parent constructor with super.new(...). Practise writing code that initialises both inherited and new attributes.

KEY IDEAS AND APPENDIX CONVENTIONS

Class and attributes

Use class and endclass, with private / public where the question needs access control.

Constructor

A public procedure named new initialises the object when created.

Inheritance

Subclass header uses inherits; super.new(...) calls the parent constructor to initialise inherited state.

EXAM-STYLE PSEUDOCODE

```
class Device
    private label
    public procedure new(givenLabel)
        label = givenLabel
    endprocedure
endclass

class Sensor inherits Device
    private limit
    public procedure new(labelIn,limitIn)
        super.new(labelIn)
        limit = limitIn
    endprocedure
endclass
```

MODEL / 2 OF 4

Construct a Sensor with inherited state

A complete worked model: follow the code, then track each intermediate state before the final output.

```
class Sensor inherits Device
    private limit
    public procedure new(labelIn,limitIn)
        super.new(labelIn)
        limit = limitIn
    endprocedure
    public function tooHigh(value)
        return value > limit
    endfunction
endclass
s = new Sensor("S2",50)
```

WORKED TRACE / THINK ALOUD

1. new Sensor("S2",50) calls Sensor.new.
2. super.new("S2") initialises parent Device label="S2".
3. Sensor stores limit=50 as its own private attribute.
4. s.tooHigh(60) returns true; 50 or lower returns false.

TRY / 3 OF 4

Trace, write, explain

Attempt the three original exam-style questions independently. The next page contains indicative marking points, so do not turn over until you finish.

BLACK PEN / ANSWER BEFORE TURNING THE PAGE

QUESTION 1 / 3 MARKS

Name the three OCR-style terms used to declare a subclass, a constructor and a call to the parent constructor.

QUESTION 2 / 5 MARKS

Trace creation of Sensor("S2",50): state the two stored attributes and the results of tooHigh(60), tooHigh(50), and tooHigh(12).

QUESTION 3 / 6 MARKS

Write an OCR-style subclass AlertSensor that inherits Sensor. Add a private message, pass label and limit to its parent constructor, store message, and provide a public getMessage() function.

CHECK / 4 OF 4

Self-mark and correct

Award only marks supported by your solution. Correct one error using green pen and explain what changed. The points below are not official OCR mark schemes.

GREEN PEN / SELF-MARK / INDICATIVE POINTS

Q1 / 3 MARKS

inherits [1]; new [1]; super.new(...) [1].

Q2 / 5 MARKS

Label S2 is set by parent [1]; limit 50 set by subclass [1]; tooHigh(60) true [1]; tooHigh(50) false [1]; tooHigh(12) false [1].

Q3 / 6 MARKS

Correct class name/inherits Sensor [1]; private message [1]; constructor new has three arguments [1]; super.new(labelIn,limitIn) [1]; assign message from supplied argument [1]; public getter correctly returns message [1].

EXAMPLE ALGORITHM / AN ACCEPTABLE SOLUTION

```
class AlertSensor inherits Sensor
    private message
    public procedure new(limLabel, lim, msg)
        super.new(limLabel, lim)
        message = msg
    endprocedure
    public function getMessage()
        return message
    endfunction
endclass
```

CORRECT ONE ERROR IN GREEN AND EXPLAIN THE CHANGE.